



MANGALORE UNIVERSITY



**IMJ INSTITUTE OF SCIENCE & COMMERCE
MOODLAKATTE, KUNDAPURA**

2024-25

**PROJECT TITLE
ART GALLERY WEB APPLICATION**

A PROJECT REPORT

Submitted by

Nethravathi	U05IM22S0031
Ananya	U05IM22S0003
Ayeshatul Sana	U05IM22S0007

***In partial fulfilment for the award of the degree of*
BACHELOR OF COMPUTER APPLICATIONS**

Under the guidance of

Mrs. Swarna Rani U P
Lecturer
(Department of Computer Application)

Submitted To:
Mangalore University in partial fulfilment of the
Requirement
For the degree of Computer Applications



MANGALORE UNIVERSITY



IMJ INSTITUTE OF SCIENCE & COMMERCE
MOODLAKATTE, KUNDAPURA

Department of Computer Application

Certificate

Certified that the Project Work Entitled

ART GALLERY WEB APPLICATION

.....is a
benefited work carried out by

Nethravathi
Reg No: U05IM22S0031

Ananya
Reg No: U05IM22S0003

Ayeshatul Sana
Reg No: U05IM22S0007

in partial fulfilment for the award of degree of Bachelor of Computer Applications of the Mangalore University during the year 2024-25 The project report has been approved as it satisfies the academic requirements of project work prescribed for the Bachelor of Computer Applications.

*Signature of the
of the Project Guide*

Signature of the HOD

*Signature
Principal*

Examiners:

1.

2

DECLARATION

We Nethravathi(U05IM22S0031), Ananya(U05IM22S0003), Ayeshatul Sana(U05IM22S0007), do hereby declare that the project entitled “ART GALLERY WEB APPLICATION” is the original work carried out by us during the sixth semester BCA, submitted to Mangalore University, for the partial fulfillment of the requirement of Bachelor of Computer Applications.

We also declare this project has not been previously submitted for any other degree or award.

Place : Moodlakatte

Name :

Date :

Nethravathi

Ananya

Ayeshutal Sana

DECLARATION

I Nethravathi (RegNo: U05IM22S0031), do hereby declare that the project entitled “ART GALLERY WEB APPLICATION” is the original work carried out by me and my teammates during the sixth semester BCA, submitted to Mangalore University, for the partial fulfillment of the requirement of Bachelor of Computer Applications.

I also declare this project has not been previously submitted for any other degree or award.

Place : Moodlakatte

Name : Nethravathi

Date :

Reg No:U05IM22S0031

DECLARATION

I Ayeshatul Sana (Regno:U05IM22S0007), do hereby declare that the project entitled “ART GALLERY WEB APPLICATION” is the original work carried out by me and my teammates during the sixth semester BCA, submitted to Mangalore University, for the partial fulfillment of the requirement of Bachelor of Computer Applications.

I also declare this project has not been previously submitted for any other degree or award.

Place :Moodlakatte

Name: Ayeshatul Sana

Date :

Reg No:U05IM22S0007

DECLARATION

I Ananya (RegNo:U05IM22S0003), do hereby declare that the project entitled “ART GALLERY WEB APPLICATION” is the original work carried out by me and my teammates during the sixth semester BCA, submitted to Mangalore University, for the partial fulfillment of the requirement of Bachelor of Computer Applications.

I also declare this project has not been previously submitted for any other degree or award.

Place : Moodlakatte

Name : Ananya

Date :

Reg No:U05IM22S0003

ACKNOWLEDGEMENT

The magnitude of this project demanded the cooperation and assistance of many individuals, and by the grace of God, we have been fortunate to receive support throughout the process of our project work.

We would like to express our sincere gratitude to our Principal, Dr. Prathibha M. Patel, for providing us with the necessary facilities to carry out this project.

Our heartfelt thanks to Mrs. Swarna Rani U.P, Head of the Department of Computer Application, IMJISC, Moodlakatte, for permitting us to undertake this project.

We are deeply grateful to our internal guide, Mrs. Swarna Rani U.P, Department of Computer Application, IMJISC, Moodlakatte, for her invaluable guidance and support throughout the course of our work.

We would also like to thank all the teaching and non-teaching staff members of the Computer Application Department, IMJISC, for their cooperation and constant encouragement, which greatly contributed to the successful completion of our project.

Finally, we extend our heartfelt gratitude to all those who helped us directly or indirectly in making this project a success.

- Nethravathi
- Ananya
- Ayeshatul sana

INDEX

Serial No	Title	Page No
1.	Introduction	1-3
2.	System Specification	4-5
3.	Requirement Analysis Document	6-9
4.	Methodology	10-13
5.	Objectives	14-15
6.	System Design	16-18
7.	Data Structure	19-21
8.	Sample Code	22-34
9.	Testing	35-36
10.	Sample Input and Output Screens	37-40
11.	Limitations	41-42
12.	Future Scope	43-45
13.	Conclusion	46-47
14.	Bibliography	48-49

INTRODUCTION

INTRODUCTION

Title of the project

ART GALLERY WEB APPLICATION

Project category

Web Application

OVERVIEW

The project “**Artistic Avenue: Online Art Gallery and E-Commerce Platform**” is a web-based application designed to connect artists and art enthusiasts through a modern digital marketplace. It allows users to browse, search, view, and purchase a wide range of artworks categorized into different styles and forms. The platform supports both user and admin functionalities, offering an interactive and immersive art shopping experience.

Art e-commerce platforms are essential in today's digital age, enabling artists to reach global audiences and giving customers the convenience of exploring and purchasing unique artwork from anywhere. Artistic Avenue bridges the gap between creativity and commerce by offering a streamlined experience for both buyers and sellers

Key features of the Artistic Avenue Web App may include:

1. **User Registration:** Visitors can create an account to explore and purchase artworks.
2. **User Authentication:** Secure login system ensures only authorized access.
3. **Artwork Browsing:** Users can view art by categories and access detailed information for each piece.
4. **Cart and Checkout System:** Customers can add artworks to a cart, proceed to checkout, and finalize their purchases.
5. **Favorites:** Users can save favorite artworks for future viewing or buying.
6. **Order History:** Buyers can view their past purchases in the order history section.
7. **Search Functionality:** An intelligent search system with live suggestions helps users find specific art.

8. **Admin Dashboard:** Admins can manage product listings, view user orders, and update the catalog.
9. **Logout and Session Control:** Users can securely log out and manage their session activities.
10. **Modern UI/UX:** Responsive layout and user-friendly design using HTML, CSS, PHP, and images for an engaging experience.

Use Cases of Art gallery Web App:

1. **Art Sales:** Artists can showcase and sell their work to a broader audience.
2. **Art Discovery:** Art lovers can explore trending, classic, and new pieces effortlessly.
3. **Art Investment:** Collectors can purchase art for investment and ownership.
4. **Digital Exhibitions:** Allows for virtual exhibitions, making art accessible to all.
5. **Admin Management:** Streamlined admin interface for inventory and order handling.

SYSTEM SPECIFICATION

SYSTEM SPECIFICATION

HARDWARE REQUIREMENTS

In hardware requirement we require all those components which will provide the platform for the development of the project. The minimum hardware required for the development of this project is as follows:-

- Processor: Intel Core i5 or higher
- RAM: Minimum 8GB
- Storage: At least 256GB SSD
- Internet connection
- Hard Disk:
- Input device: keyboard, mouse

SOFTWARE REQUIREMENTS

Software can be defined as a programs which run on our computer. it acts as a role in the vehicle. it provides the relationship between the human and computer. it is very important to run software to function the computer. The software needed to the project is as follows:-

- Database: MySQL
- Web Server: Apache
- Code Editor: VS Code or PHP Storm
- Browser: Chrome, Firefox, or Edge
- Operating System: Windows

REQUIREMENT ANALYSIS DOCUMENT

REQUIREMENTS ANALYSIS DOCUMENT

The purpose of this document is to outline the requirements for the **Online Art Gallery Web Application – ArtVista**. This includes **functional and non-functional requirements, user roles, and system specifications** to ensure a clear understanding of what is necessary for successful development and deployment.

Functional Requirements:

1. User Registration & Authentication

- Users (both buyers and artists) must be able to register and log in securely.
- Password recovery and reset options should be available.
- Unique usernames and email validation are required during registration.

2. User Roles

- **Buyer:** Can browse artworks, manage cart, place orders, and view purchase history.
- **Artist:** Can upload artworks, manage their portfolio, view order requests, and track sales.
- **Admin:** Can manage users, artwork listings, approve or reject uploads, and oversee orders.

3. Artwork Management

- Artists can add new artworks with title, description, category, image, and price.
- Artists can edit or remove existing listings.
- Admin can approve or reject submitted artwork before publishing.

4. Artwork Browsing and Search

- Users can browse artworks by categories such as Paintings, Sculptures, Digital Art, etc.
- Search functionality based on keywords, artist names, price range, or tags.
- Filters for sorting by newest, popularity, and price.

5. Product Page

- Each artwork has a dedicated page with detailed description, artist info, high-resolution images, and an "Add to Cart" button.

6. Shopping Cart and Checkout

- Buyers can add multiple items to a cart.
- Checkout process includes entering personal and payment details, confirming order, and viewing invoice.
- Buyers receive email confirmation upon successful purchase.

7. Order Management

- Artists can view all received orders and update order status (e.g., pending, shipped, completed).
- Buyers can track their orders and view past purchases.

8. Admin Dashboard

- Admin can manage user accounts (add/remove/ban users), review artwork submissions, and oversee order status.
- Access to analytics such as number of sales, most viewed artworks, top artists, etc.

9. Email Notifications

- Users receive emails for order confirmation, shipping updates, and password reset links.
- Artists are notified when their artwork is sold.

Non-Functional Requirements

1. Performance

- The application must load pages quickly and process transactions efficiently.
- Able to handle high traffic and simultaneous users without crashing.

2. Security

- All transactions must be protected with HTTPS and SSL encryption.
- User data, including passwords and payment details, must be securely stored.
- Comply with standard data protection laws (e.g., GDPR).

3. Usability

- Clean, intuitive, and responsive design.
- Easy navigation for all types of users (tech-savvy or not).

- Accessible to people with disabilities (screen reader support, contrast options, etc.).

4. Reliability

- The system should have minimal downtime and fast recovery in case of failures.
- Regular backups and error handling mechanisms should be in place.

5. Compatibility

- Fully functional across major browsers (Chrome, Firefox, Edge, Safari).
- Responsive design for tablets and smartphones (iOS and Android).
- Platform-independent—accessible from Windows, macOS, Linux

METHODOLOGY

METHODOLOGY

1. Introduction

Objective: The primary objective of this project was to develop a fully functional online art gallery platform where artists can upload and manage their artwork while buyers can explore, compare, and purchase art seamlessly. The system was developed using HTML, CSS, PHP, and MySQL with a focus on e-commerce capabilities.

Scope: The scope of the project included artist and buyer registration, artwork listing and categorization, cart management, secure order processing, responsive design, and an admin panel for product and order management.

2. Project Planning

Objective Setting: The main goals of the project were to deliver an intuitive, secure, and visually appealing web application that enables users to explore art, make purchases, and manage their content efficiently.

Requirements Gathering: Requirements were collected through team discussions and user analysis. User personas were defined for both buyers and artists to outline essential features such as product browsing, uploading artwork, checkout, and admin controls.

Technology Selection: PHP was chosen for backend development due to its efficiency with MySQL for relational database handling. HTML and CSS were used for designing a responsive and aesthetic frontend. JavaScript was included for enhancing interactivity and validation. Bootstrap was adopted to accelerate UI development with responsive design.

3. Design and Architecture

System Architecture: The system follows a traditional web-based client-server model. The client-side handles the user interface, while the server processes requests and manages data stored in the MySQL database.

Database Design: The database schema includes the following primary tables: users (for buyers and artists), artworks, orders, cart, and admin. Relationships were designed to maintain referential integrity and to ensure efficient data retrieval and transaction processing.

User Interface Design: The UI was crafted to be clean and user-friendly with clear navigation, image-based artwork previews, category filtering, and a responsive layout to support mobile and desktop viewing.

4. Implementation

Backend Development: PHP was used to handle core functionalities including user registration, login, file uploads for artwork, cart operations, order placement, and admin operations such as adding, editing, or removing products. Security practices like input sanitization and password hashing were implemented.

Frontend Development: HTML and CSS were used to create structured pages such as the homepage, product listings, cart view, and user dashboards. JavaScript was employed for form validation, dynamic cart updates, and interactive features like image hover effects.

Integration: The frontend and backend were seamlessly connected using PHP form handling and server-side scripting. Database operations were validated through AJAX requests in selected features to enhance responsiveness without full-page reloads.

5. Testing

Testing Strategy: The testing process included unit testing of PHP functions, database query validation, integration testing of user interactions across the interface, and UI/UX testing on multiple devices and browsers.

Tools Used: Manual testing was primarily used with inspection tools from browsers like Chrome Developer Tools. SQL query validation was done using phpMyAdmin. Form validations were tested using JavaScript and cross-browser testing ensured compatibility.

Test Results: Testing revealed and resolved issues like incorrect cart calculations, missing image uploads, and input validation bugs. All core functionalities were confirmed to work correctly before deployment.

6. Evaluation and Results

Performance Evaluation: The application efficiently handled multiple users performing simultaneous actions such as browsing artwork and adding to cart. Performance tuning was considered for future scaling by optimizing SQL queries and enabling caching.

User Feedback: Feedback from test users and peers indicated satisfaction with the overall interface and functionality. Suggestions included adding features like artist reviews, artwork search by price range, and wishlist functionality.

Achievement of Objectives: The project successfully met its primary objectives—offering an online platform for artists to display their work and for buyers to explore and securely purchase artwork.

OBJECTIVES

OBJECTIVES

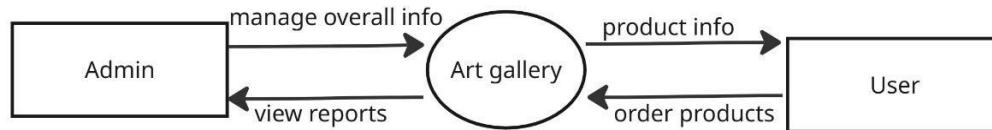
The primary objective of **Artistic Avenue** is to create a centralized online space where art can be discovered, appreciated, and purchased with ease. This mission is supported by the following specific objectives:

1. **Promote Art Accessibility:** Provide a digital platform where diverse art forms are easily accessible to the public.
2. **Empower Artists:** Offer artists a space to present their work and reach new customers without needing a physical gallery.
3. **Simplify Art Buying:** Streamline the process of browsing, selecting, and purchasing art online.
4. **Enhance User Experience:** Create a visually appealing and easy-to-navigate interface that encourages exploration and engagement.
5. **Provide Order Management:** Enable users to track their past orders and favorite products.
6. **Ensure Secure Transactions:** Use secure authentication and order management to protect users' data and privacy.
7. **Enable Admin Control:** Allow admins to manage the product inventory and track sales efficiently.
8. **Encourage Repeated Engagement:** Provide favorites and history features to keep users coming back.
9. **Optimize Performance:** Ensure the site loads quickly and functions well across all devices and screen sizes.
10. **Foster Art Community:** Promote interaction between users, building a digital community of art lovers and creators.
11. **Empower Artists:** Offer artists a space to present their work and reach new customers without needing a physical gallery.

SYSTEM DESIGN

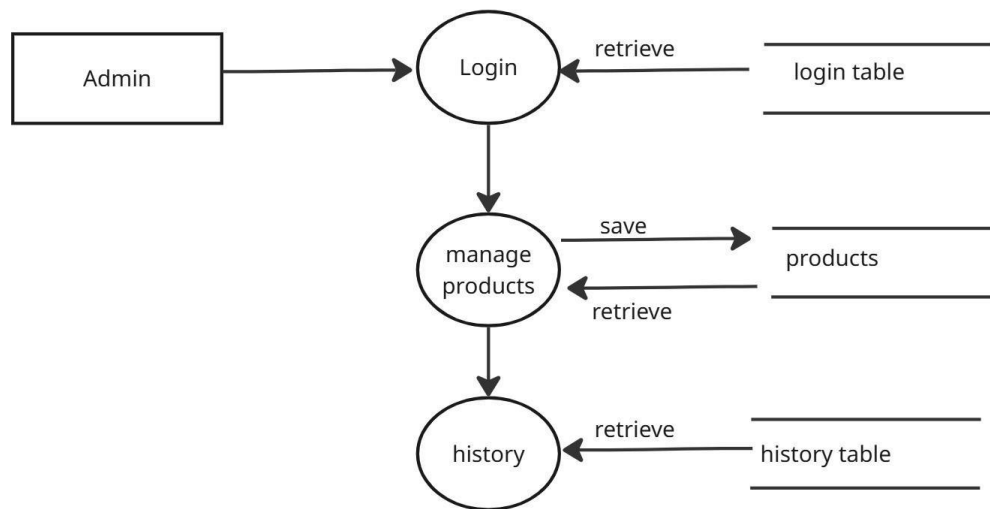
SYSTEM DESIGN

5.1. CFD(Context Flow Diagram)

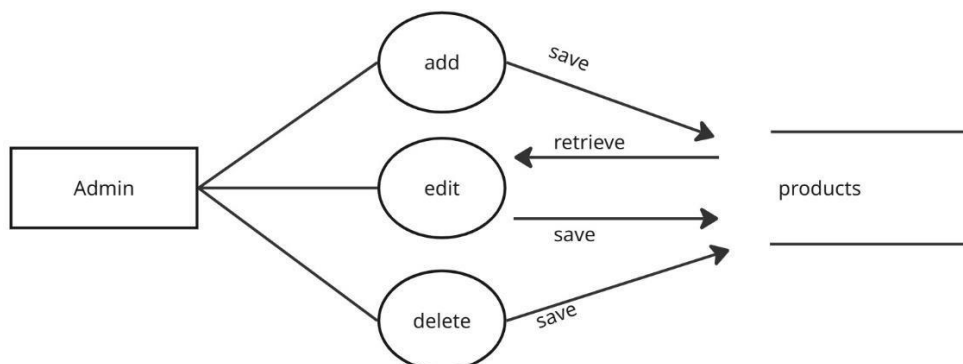


5.2. DFD(DATA FLOW DIAGRAM)

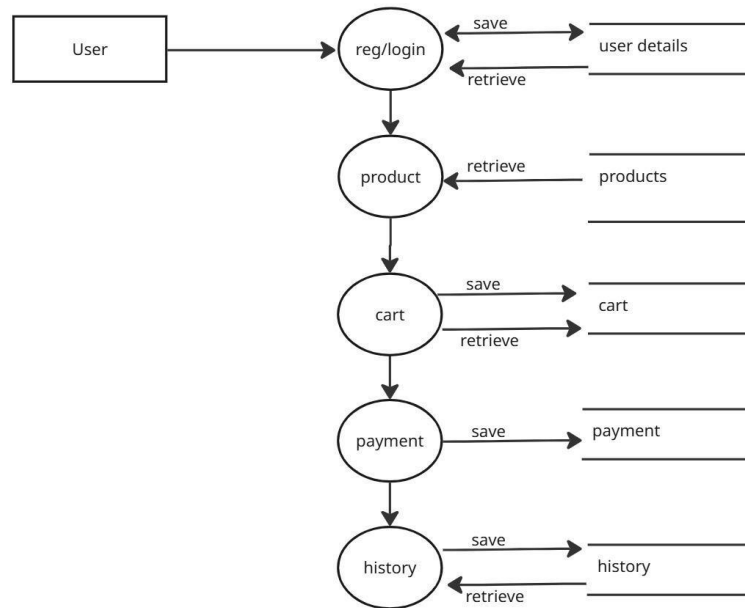
5.3. Admin-Level:0



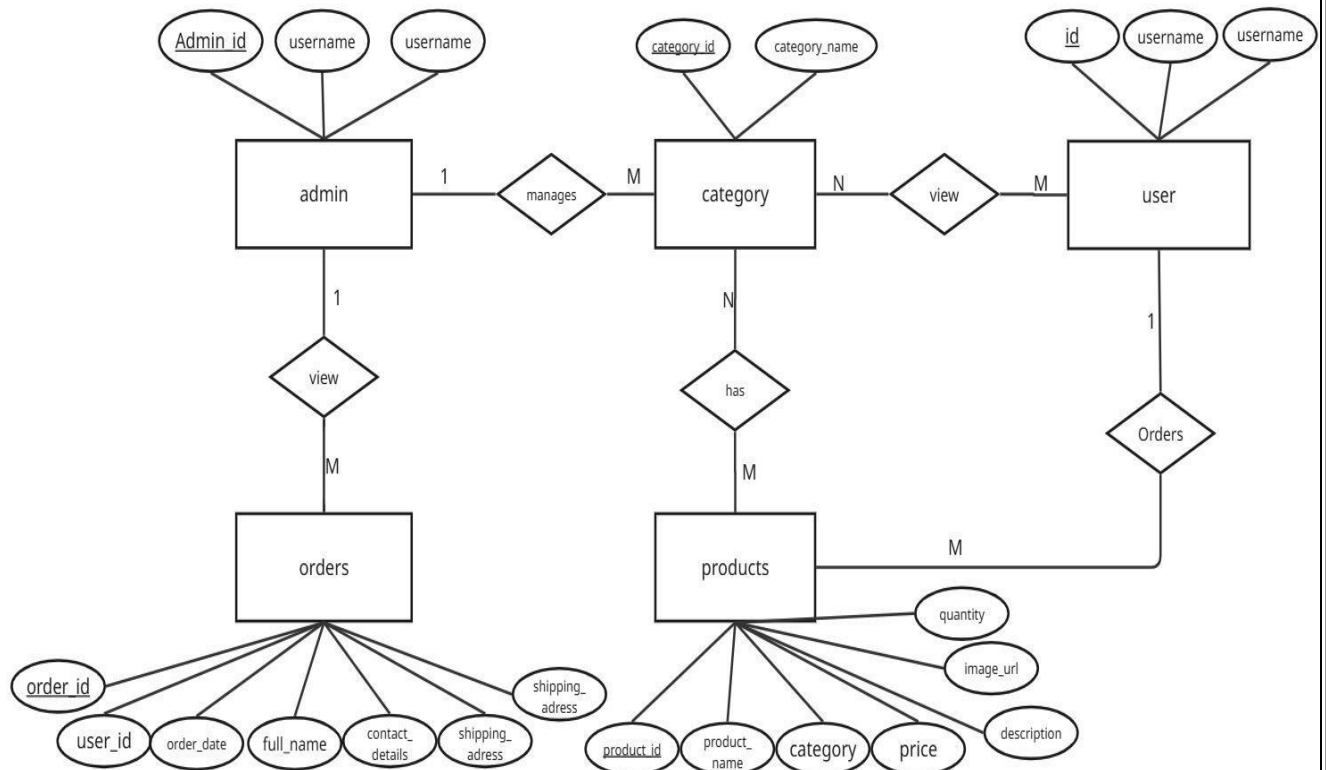
Level 1:



DFD:USER



5.4. ER DIAGRAM



DATA STRUCTURE

Database Tables: artistic_avenue

Table: admin

Column Name	Data Type	Description	Size	Null
Admin_id	INT	Unique user ID	11	NO
Username	VARCHAR	Username of the user	50	NO
Password	VARCHAR	password	255	NO

Table: products

Column Name	Data Type	Description	Size	Null
product_id	INT	Unique product ID	11	NO
product_name	VARCHAR	Name of the product	100	NO
category	ENUM	Product category (Sculpture, Paintings, Digital Art)	-	NO
price	DECIMAL	Product price	10,2	NO
description	TEXT	Product description	-	YES
image_url	VARCHAR	Image URL of the product	255	NO
quantity	INT	Available quantity	11	NO

Table: users

Column Name	Data Type	Description	Size	Null
user_id	INT	Unique user ID	11	NO
username	VARCHAR	Username of the user	50	NO
password_hash	VARCHAR	Hashed password	255	NO
is_admin	TINYINT	Admin status (0/1)	1	NO

Table: favorites

Column Name	Data Type	Description	Size	Null
favorite_id	INT	Unique favorite ID	11	NO
user_id	INT	User ID from users	11	NO
product_id	INT	Product ID from products	11	NO

Table: orders

Column Name	Data Type	Description	Size	Null
order_id	INT	Unique order ID	11	NO
user_id	INT	User ID from users	11	NO
order_date	DATETIME	Order date and time	-	NO
full_name	VARCHAR	Full name of the customer	100	NO
contact_details	VARCHAR	Contact information	100	NO
shipping_address	VARCHAR	Shipping address	255	NO
total_amount	DECIMAL	Total order amount	10,2	NO

Table: order_items

Column Name	Data Type	Description	Size	Null
order_item_id	INT	Unique order item ID	11	NO
order_id	INT	Order ID from orders	11	NO
product_id	INT	Product ID from products	11	NO
Quantity	INT	Quantity of the product	11	NO
price_each	DECIMAL	Price of each item	10,2	NO

Table:cart

Column Name	Data Type	Description	Size	Allow Null Value
cart_id	INT	Primary Key, Auto Increment	11	No
user_id	INT	Foreign Key (users)	11	No
product_id	INT	Foreign Key (products)	11	No
quantity	INT	Number of Items	11	No
created_at	TIMESTAMP	Record Creation Time	-	No
updated_at	TIMESTAMP	Last Update Time	-	No

Program sample code

PROGRAM SAMPLE CODE

Register.php

```
<?php
// register.php
require_once "includes/config.php";

// If the user is already logged in, redirect to index
if (isset($_SESSION['user_id'])) {
    header("Location: index.php");
    exit;
}

$errors = [];
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $username = trim($_POST['username']);
    $password = trim($_POST['password']);

    // Basic validations
    if (empty($username) || empty($password)) {
        $errors[] = "All fields are required.";
    } else {
        // Check if username exists
        $stmt = $pdo->prepare("SELECT user_id FROM users WHERE
username = :username");
        $stmt->execute(['username' => $username]);
        if ($stmt->rowCount() > 0) {
            $errors[] = "Username already taken. Already registered? <a
href='login.php'>Login here</a>.";
        } else {
            // Hash the password and insert user
            $password_hash = password_hash($password,
PASSWORD_DEFAULT);
            $stmt = $pdo->prepare("INSERT INTO users (username,
password_hash) VALUES (:username, :password_hash)");
            $stmt->execute([
                'username' => $username,
                'password_hash' => $password_hash
            ]);
            // Redirect to login page with a success message
            header("Location: login.php?registered=1");
            exit;
        }
    }
}
```

```
}  
?>
```

```
<?php include "includes/header.php"; ?>
```

```
<div class="container mt-4">  
  <h2>Register</h2>  
  <?php if (!empty($errors)): ?>  
    <div class="alert alert-danger">  
      <?php foreach ($errors as $err) : ?>  
        <p><?php echo $err; ?></p>  
      <?php endforeach; ?>  
    </div>  
  <?php endif; ?>  
  <form method="post" action="register.php" class="col-md-6">  
    <div class="mb-3">  
      <label for="username" class="form-label">Username</label>  
      <input type="text" name="username" id="username" class="form-control" required />  
    </div>  
    <div class="mb-3">  
      <label for="password" class="form-label">Password</label>  
      <input type="password" name="password" id="password" class="form-control" required />  
    </div>  
    <button type="submit" class="btn btn-primary">Register</button>  
  </form>  
  <p class="mt-3">Already registered? <a href="login.php">Login here</a>.</p>  
</div>
```

```
<?php include "includes/footer.php"; ?>
```

[login.php](#)

```
<?php  
// login.php  
require_once "includes/config.php";  
  
// If already logged in, redirect  
if (isset($_SESSION['user_id'])) {  
  header("Location: index.php");  
  exit;
```

```

    }

    $errors = [];
    $registered = isset($_GET['registered']);

    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        $username = trim($_POST['username']);
        $password = trim($_POST['password']);

        if (empty($username) || empty($password)) {
            $errors[] = "All fields are required.";
        } else {
            // Check user
            $stmt = $pdo->prepare("SELECT user_id, password_hash, is_admin
FROM users WHERE username = :username");
            $stmt->execute(['username' => $username]);
            $user = $stmt->fetch(PDO::FETCH_ASSOC);

            if ($user && password_verify($password, $user['password_hash'])) {
                // Correct password
                $_SESSION['user_id'] = $user['user_id'];
                $_SESSION['is_admin'] = $user['is_admin'];
                header("Location: index.php");
                exit;
            } else {
                $errors[] = "Invalid username or password.";
            }
        }
    }
}
?>
<?php include "includes/header.php"; ?>
<?php include "includes/navbar.php"; ?>

<div class="container mt-4">
    <h2>Login</h2>
    <?php if ($registered): ?>
        <div class="alert alert-success">
            Registration successful! Please log in.
        </div>
    <?php endif; ?>
    <?php
    if (!empty($errors)) {
        echo '<div class="alert alert-danger">';
    }
}

```



```

        foreach ($errors as $err) {
            echo "<p>$err</p>";
        }
        echo '</div>';
    }
?> <form method="post" action="login.php" class="col-md-6">
    <div class="mb-3">
        <label for="username" class="form-label">Username</label>
        <input type="text" name="username" id="username" class="form-
control" required />
    </div>
    <div class="mb-3">
        <label for="password" class="form-label">Password</label>
        <input type="password" name="password" id="password"
class="form-control" required />
    </div>
    <button type="submit" class="btn btn-primary">Login</button>
</form>
</div>

<?php include "includes/footer.php"; ?>

```

Products.php

```

<?php
require_once "includes/config.php";

// Ensure the user is logged in and valid
if (!isset($_SESSION['user_id']) || empty($_SESSION['user_id'])) {
    header("Location: login.php");
    exit;
}

// Verify user exists
$stmt = $pdo->prepare("SELECT user_id FROM users WHERE user_id =
:uid");
$stmt->execute(['uid' => $_SESSION['user_id']]);
<form method="post" action="login.php" class="col-md-6">
    <div class="mb-3">
        <label for="username" class="form-label">Username</label>
        <input type="text" name="username" id="username" class="form-
control" required />
    </div>

```

```

        <div class="mb-3">
            <label for="password" class="form-label">Password</label>
            <input type="password" name="password" id="password"
class="form-control" required />
        </div>
        <button type="submit" class="btn btn-primary">Login</button>
    </form>
</div>

<?php include "includes/footer.php"; ?>

```

Products.php

```

<?php
require_once "includes/config.php";

// Ensure the user is logged in and valid
if (!isset($_SESSION['user_id']) || empty($_SESSION['user_id'])) {
    header("Location: login.php");
    exit;
}

// Verify user exists
$stmt = $pdo->prepare("SELECT user_id FROM users WHERE user_id = :uid");
$stmt->execute(['uid' => $_SESSION['user_id']]);
if (!$stmt->fetch()) {
    unset($_SESSION['user_id']);
    header("Location: login.php");
    exit;
}

// Retain submitted form data
$full_name = $phone_number = $shipping_address = "";
$order_summary = [];
$order_items_summary = [];
$success = false;
// Handle "Buy Now" action
if ($_SERVER['REQUEST_METHOD'] === 'POST' &&
isset($_POST['action']) && $_POST['action'] === 'buy_now') {
    $_SESSION['cart'] = [];
    $pid = (int) $_POST['product_id'];
    $_SESSION['cart'][$pid] = 1;
}

```

```

}
// Update cart quantities
if ($_SERVER['REQUEST_METHOD'] === 'POST' &&
isset($_POST['checkout_submit'])) {
    if (isset($_POST['quantities']) && is_array($_POST['quantities'])) {
        foreach ($_POST['quantities'] as $pid => $qty) {
            $_SESSION['cart'][$pid] = (int)$qty;
        }
    }
}

// Fetch cart information
$cart_products = [];
$total_price = 0;
if (!empty($_SESSION['cart'])) {
    $in_placeholders = rtrim(str_repeat('?', count($_SESSION['cart']), ','), ',');
    $stmt = $pdo->prepare("SELECT * FROM products WHERE product_id
IN ($in_placeholders)");
    $stmt->execute(array_keys($_SESSION['cart']));
    $cart_products = $stmt->fetchAll(PDO::FETCH_ASSOC);

    foreach ($cart_products as $cp) {
        $pid = $cp['product_id'];
        $qty = isset($_SESSION['cart'][$pid]) ? $_SESSION['cart'][$pid] : 0;
        $total_price += $cp['price'] * $qty;
    }
}

// Process checkout form
if (isset($_POST['checkout_submit'])) {
    $full_name = trim($_POST['full_name']);
    $phone_number = trim($_POST['phone_number']);
    $shipping_address = trim($_POST['shipping_address']);
    $errors = [];
    if (empty($full_name) || empty($phone_number) || empty($shipping_address))
    {
        $errors[] = "All fields are required.";
    } elseif (empty($cart_products)) {
        $errors[] = "Your cart is empty.";
    } else {
        // Insert order details
        $stmt = $pdo->prepare("INSERT INTO orders (user_id, full_name,
contact_details, shipping_address, total_amount)
VALUES (:uid, :fname, :phone, :ship, :total)");

```

```

$stmt->execute([
    'uid' => $_SESSION['user_id'],
    'fname' => $full_name,
    'phone' => $phone_number,
    'ship' => $shipping_address,
    'total' => $total_price
]);
$order_id = $pdo->lastInsertId();

// Insert order items and update stock
foreach ($cart_products as $cp) {
    $pid = $cp['product_id'];
    $qty = $_SESSION['cart'][$pid];
    $price_each = $cp['price'];

    $stmt_item = $pdo->prepare("INSERT INTO order_items (order_id,
product_id, quantity, price_each)
                                VALUES (:oid, :pid, :qty, :price)");
    $stmt_item->execute([
        'oid' => $order_id,
        'pid' => $pid,
        'qty' => $qty,
        'price' => $price_each
    ]);

    $stmt_update = $pdo->prepare("UPDATE products SET quantity =
quantity - :qty WHERE product_id = :pid");
    $stmt_update->execute([
        'qty' => $qty,
        'pid' => $pid
    ])
}

$order_items_summary[] = [
    'product_name' => $cp['product_name'],
    'quantity' => $qty,
    'price_each' => $price_each
];
}

$order_summary = [
    'order_id' => $order_id,
    'full_name' => $full_name,
    'phone_number' => $phone_number,

```

```

        'shipping_address' => $shipping_address,
        'total_amount' => $total_price
    ];

    $_SESSION['cart'] = [];
    $success = true;
}
}
?>

<?php include "includes/header.php"; ?>
<?php include "navbar2.php"; ?>

<div class="container mt-4">
    <h2>Checkout</h2>

    <?php if (!empty($errors)): ?>
        <div class="alert alert-danger">
            <?php foreach ($errors as $error): ?>
                <p><?php echo htmlspecialchars($error); ?></p>
            <?php endforeach; ?>
        </div>
    <?php endif; ?>

    <?php if ($success): ?>
        <div class="alert alert-success">🎉 Order placed successfully!</div>
        <h3>Order Summary</h3>
        <p><strong>Order ID:</strong> <?php echo
$order_summary['order_id']; ?></p>
        <p><strong>Full Name:</strong> <?php echo
htmlspecialchars($order_summary['full_name']); ?></p>

        <p><strong>Phone Number:</strong> <?php echo
htmlspecialchars($order_summary['phone_number']); ?></p>
        <p><strong>Shipping Address:</strong> <?php echo
htmlspecialchars($order_summary['shipping_address']); ?></p>
        <p><strong>Total Amount:</strong> $<?php echo
number_format($order_summary['total_amount'], 2); ?></p>

        <h4>Products Ordered:</h4>
        <ul>

```

```

        <?php foreach ($order_items_summary as $item): ?>
            <li>
                <?php echo htmlspecialchars($item['product_name']); ?>
                (Quantity: <?php echo $item['quantity']; ?>, Price Each: $<?php
echo number_format($item['price_each'], 2); ?>)
            </li>
        <?php endforeach; ?>
    </ul>
    <a href="order_history.php" class="btn btn-primary mt-3">View Order
History</a>
    <?php else: ?>
        <?php if (!empty($cart_products)): ?>
            <form action="checkout.php" method="POST" class="col-md-8">
                <h4>Order Details</h4>
                <table class="table table-bordered" id="cartTable">
                    <thead>
                        <tr>
                            <th>Product</th>
                            <th>Price</th>
                            <th>Quantity</th>
                            <th>Subtotal</th>
                        </tr>
                    </thead>
                    <tbody>
                        <?php foreach ($cart_products as $cp):
                            $pid = $cp['product_id'];
                            $currentQty = $_SESSION['cart'][$pid];
                            $subtotal = $cp['price'] * $currentQty;
                            ?>
                            <tr>
                                <td><?php echo htmlspecialchars($cp['product_name']);
?></td>
                                <td class="price" data-price="<?php echo $cp['price'];
?>">$<?php echo number_format($cp['price'], 2); ?></td>
                                <td>
                                    <input type="number" name="quantities[<?php echo $pid; ?>]"
value="<?php echo $currentQty; ?>" min="1" class="form-control quantity"
data-id="<?php echo $pid; ?>">
                                </td>
                                <td class="subtotal">$<?php echo number_format($subtotal,
2); ?></td>
                            </tr>

```

```

        <?php endforeach; ?>
    </tbody>
</table>
<h4>Total Amount: $<span id="totalAmount"><?php echo
number_format($total_price, 2); ?></span></h4>

<h4 class="mt-4">Shipping Details</h4>
<div class="mb-3">
    <label for="full_name" class="form-label">Full Name</label>
    <input type="text" name="full_name" id="full_name"
class="form-control" value="<?php echo htmlspecialchars($full_name); ?>"
required>
</div>
<div class="mb-3">
    <label for="phone_number" class="form-label">Phone
Number</label>
    <input type="text" name="phone_number" id="phone_number"
class="form-control" value="<?php echo htmlspecialchars($phone_number);
?>" required>
</div>
<div class="mb-3">
    <label for="shipping_address" class="form-label">Shipping
Address</label>
    <textarea name="shipping_address" id="shipping_address"
class="form-control" required><?php echo
htmlspecialchars($shipping_address); ?></textarea>
</div>
<button type="submit" name="checkout_submit" class="btn btn-
success">Place Order (COD)</button>
</form>
<?php else: ?>
    <p>Your cart is empty.</p>
<?php endif; ?>
<?php endif; ?>
</div>

<?php include "includes/footer.php"; ?>
<script>

document.querySelectorAll('.quantity').forEach(input => {
    input.addEventListener('input', function() {
        let qty = parseInt(this.value);
        let price =

```

```

parseFloat(this.closest('tr').querySelector('.price').getAttribute('data-price'));
    let subtotalElem = this.closest('tr').querySelector('.subtotal');

    let newSubtotal = qty * price;
    subtotalElem.textContent = '$' + newSubtotal.toFixed(2);

    let total = 0;
    document.querySelectorAll('.subtotal').forEach(sub => {
        total += parseFloat(sub.textContent.replace('$', ''));
    });

    document.getElementById('totalAmount').textContent =
total.toFixed(2);
    });
    });
</script>
}

```

Admin_login.php

```

<?php
// admin/admin_login.php
require_once "../includes/config.php";

// If admin is already logged in, redirect to dashboard
if (isset($_SESSION['is_admin']) && $_SESSION['is_admin'] == 1) {
    header("Location: admin_dashboard.php");
    exit;
}

$errors = [];
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $username = trim($_POST['username']);
    $password = trim($_POST['password']);
    // Fixed admin credentials
    $valid_admins = ['ananya', 'netra', 'sana'];
    $valid_password = 'ASN';

    if (in_array($username, $valid_admins) && $password ===
    $valid_password) {

```



```

        // Set session variables for admin access
        $_SESSION['user_id'] = $username;
        $_SESSION['is_admin'] = 1;
        header("Location: admin_dashboard.php");
        exit;
    } else {
        $errors[] = "Invalid admin credentials.";
    }
}
?>
<?php include "../includes/header.php"; ?>

<div class="container mt-4">
    <h2>Admin Login</h2>
    <?php if (!empty($errors)): ?>
        <div class="alert alert-danger">
            <?php foreach ($errors as $error): ?>
                <p><?php echo htmlspecialchars($error); ?></p>
            <?php endforeach; ?>
        </div>
    <?php endif; ?>
    <form method="POST" action="admin_login.php" class="col-md-6">
        <div class="mb-3">
            <label for="username" class="form-label">Admin Username</label>
            <input type="text" name="username" id="username" class="form-control" required />
        </div>
        <div class="mb-3">
            <label for="password" class="form-label">Admin Password</label>
            <input type="password" name="password" id="password"
class="form-control" required />
        </div>
        <button type="submit" class="btn btn-primary">Login as
Admin</button>
    </form>
</div>
<?php include "../includes/footer.php"; ?>

```

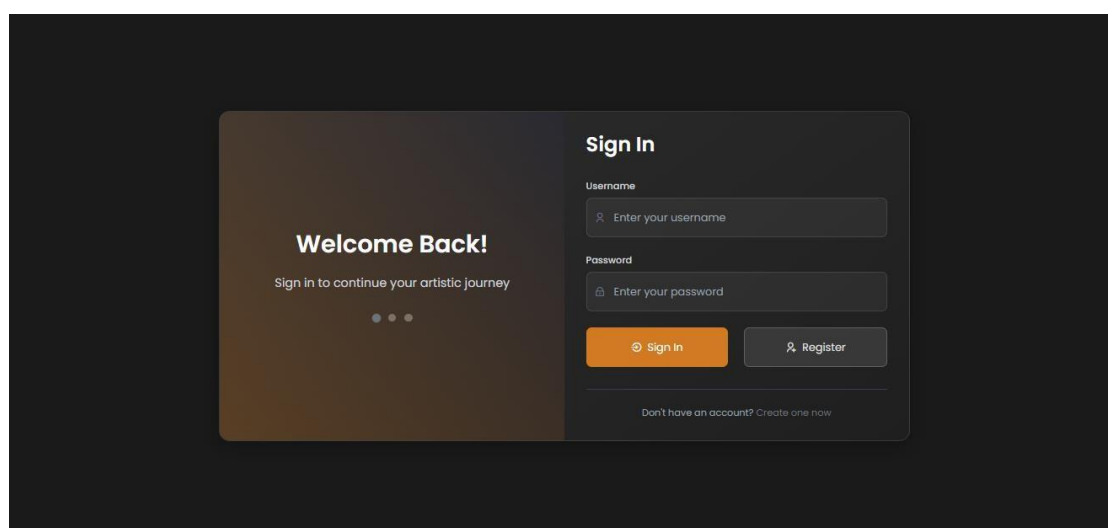
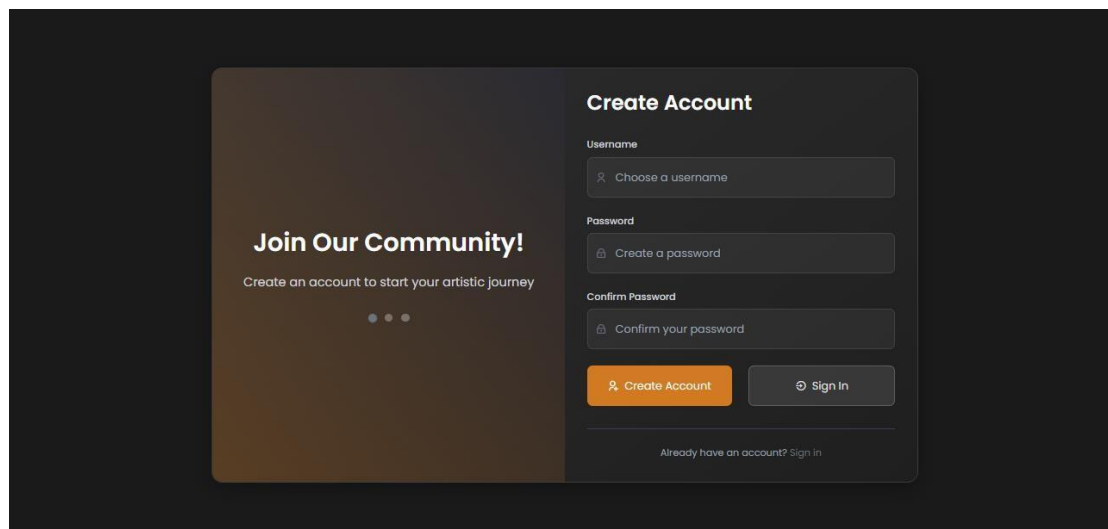
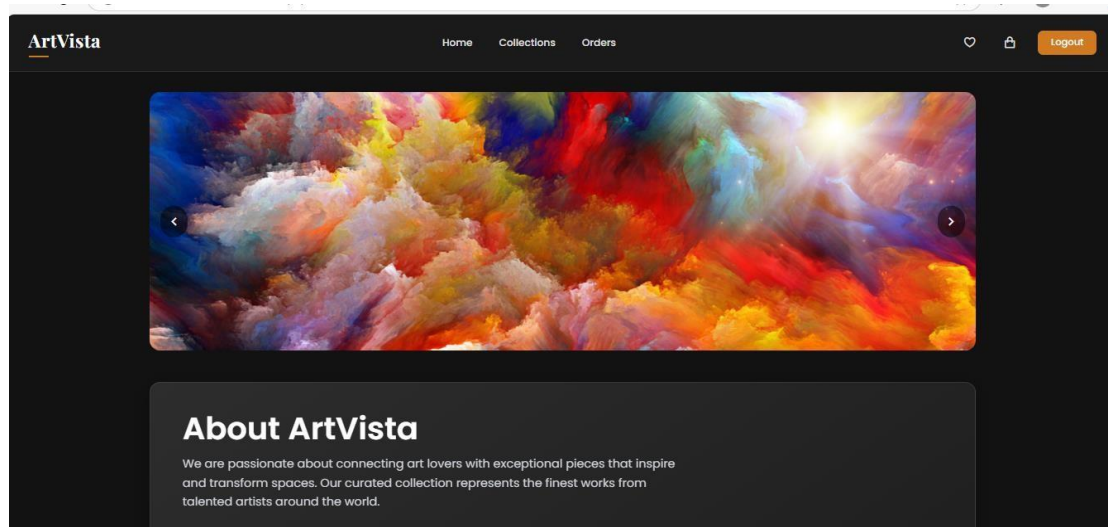
TESTING

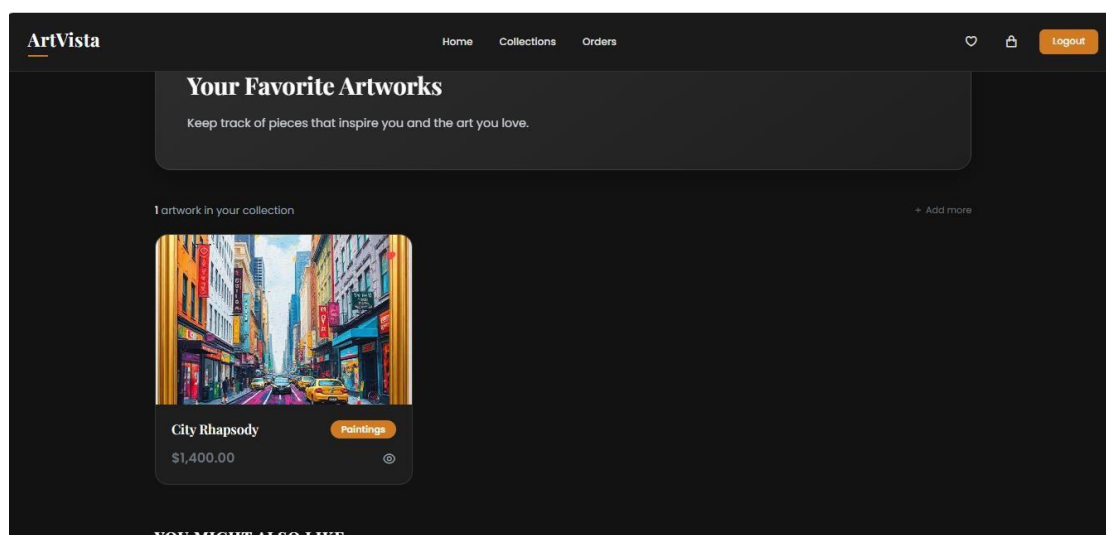
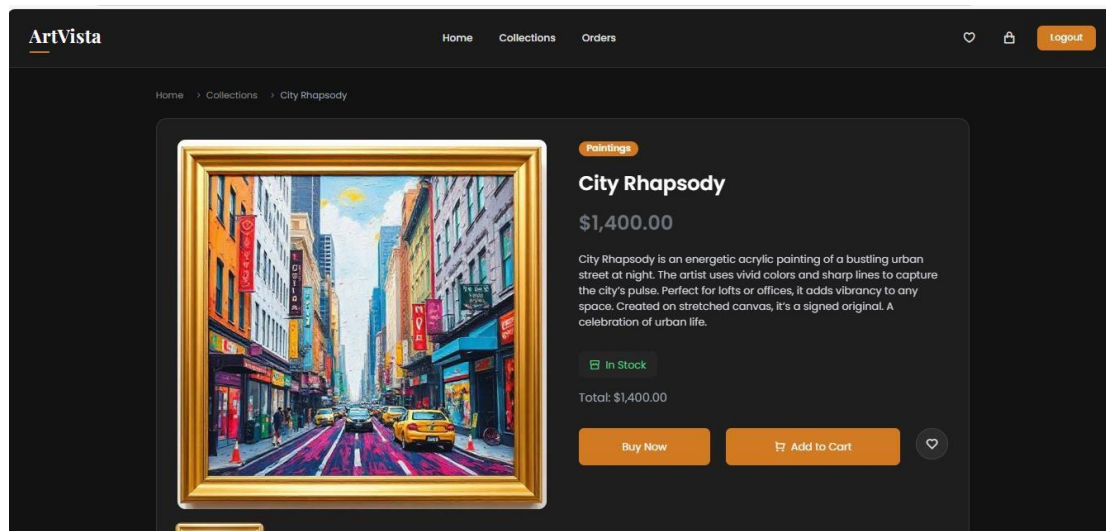
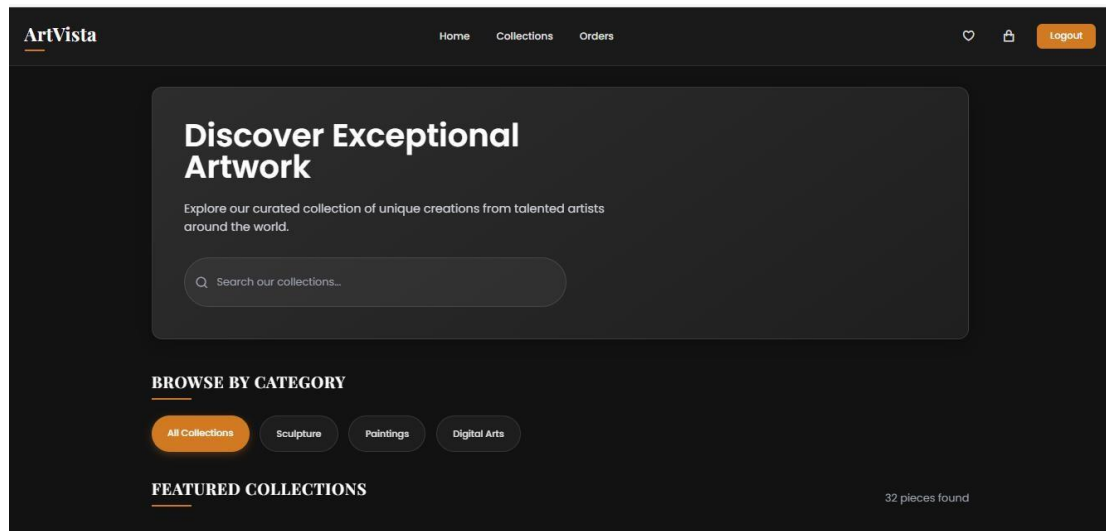
TESTING

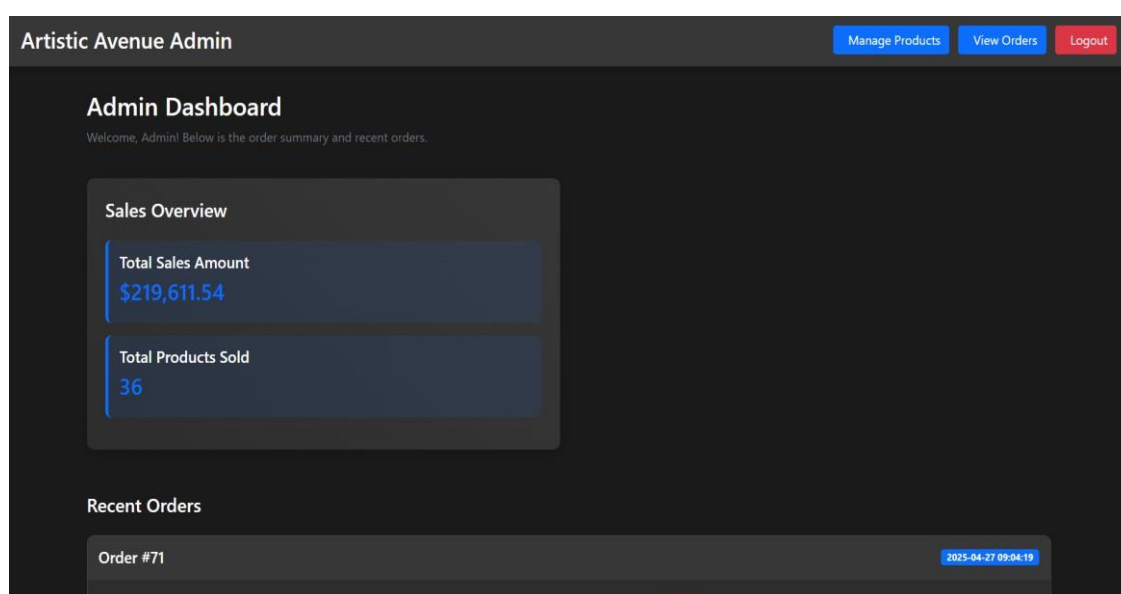
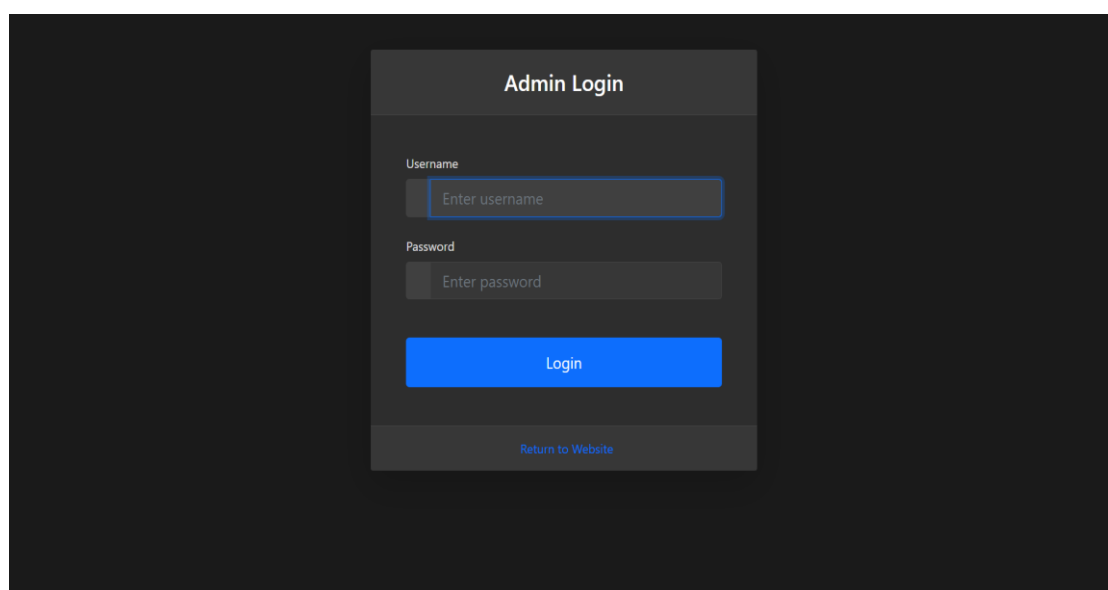
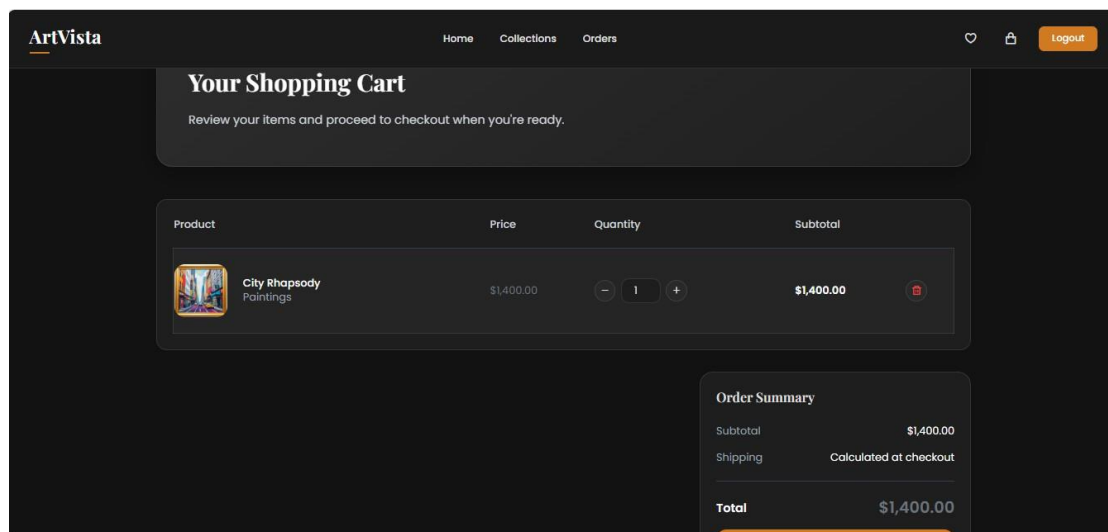
Testing the ArtVista Web Application involves a comprehensive approach to ensure that all features function correctly, the user experience is seamless, and the system is secure and reliable.

- **Functionality Testing**
Functionality Testing checks that all features like user registration, login, artwork upload, browsing, cart system, and checkout are working as expected.
- **Usability Testing**
Usability Testing ensures the website is easy to navigate, allowing both artists and buyers to perform tasks like uploading art or making purchases smoothly.
- **Performance Testing**
Performance Testing evaluates how fast and efficiently the application responds during actions like browsing, searching, or placing orders.
- **Compatibility Testing**
Compatibility Testing ensures the application works properly across different web browsers (Chrome, Firefox, Safari) and devices (desktop, tablet, mobile).
- **Security Testing**
Security Testing verifies that data is protected, passwords are encrypted, and users cannot access unauthorized areas or perform harmful actions.
- **Load Testing**
Load Testing checks how the site handles many users at once, especially during high-traffic periods like art sales or new uploads.
- **Integration Testing**
Integration Testing ensures smooth operation between features such as the database, payment system, email notifications, and user dashboards..

SAMPLE INPUT & OUTPUT SCREENS







Product Management

Add New Product

All Products

ID	Name	Category	Price	Qty	Status	Actions
39	Infinite Loop	Sculpture	\$3,200.00	24	In Stock	Edit Delete
38	Serenity Bust	Sculpture	\$4,800.00	28	In Stock	Edit Delete
37	Fractured Horizon	Sculpture	\$6,500.00	21	In Stock	Edit Delete
36	Guardian Totem	Sculpture	\$1,500.00	35	In Stock	Edit Delete
35	Dancing Shadows	Sculpture	\$2,900.00	26	In Stock	Edit Delete
34	Celestial Sphere	Sculpture	\$3,600.00	40	In Stock	Edit Delete
33	Whispering Oak	Sculpture	\$1,800.00	16	In Stock	Edit Delete
32	Urban Pulse	Sculpture	\$4,200.00	29	In Stock	Edit Delete
31	Marble Embrace	Sculpture	\$8,000.00	29	In Stock	Edit Delete

Product Management

Add New Product

All Products

ID	Name	Category	Price	Qty	Status	Actions
39	Infinite Loop	Sculpture	\$3,200.00	24	In Stock	Edit Delete
38	Serenity Bust	Sculpture	\$4,800.00	28	In Stock	Edit Delete
37	Fractured Horizon	Sculpture	\$6,500.00	21	In Stock	Edit Delete
36	Guardian Totem	Sculpture	\$1,500.00	35	In Stock	Edit Delete
35	Dancing Shadows	Sculpture	\$2,900.00	26	In Stock	Edit Delete
34	Celestial Sphere	Sculpture	\$3,600.00	40	In Stock	Edit Delete
33	Whispering Oak	Sculpture	\$1,800.00	16	In Stock	Edit Delete
32	Urban Pulse	Sculpture	\$4,200.00	29	In Stock	Edit Delete
31	Marble Embrace	Sculpture	\$8,000.00	29	In Stock	Edit Delete

LIMITATIONS

LIMITATIONS

- 1. Web-Only Access:** The platform is only accessible through web browsers and lacks a dedicated mobile application, limiting the user experience on mobile devices.
- 2. Internet Dependency:** Users must have a stable internet connection to access, browse, or purchase artwork from the platform.
- 3. Manual Payment Processing:** Payments are not automated through integrated gateways like PayPal or Stripe. Users must manually transfer funds and upload payment proof (e.g., transaction ID or screenshot). This causes delays in order verification and confirmation, increasing the risk of errors or fraudulent claims.
- 4. Limited Artwork Preview Resolution:** Due to bandwidth or hosting constraints, high-resolution previews of artworks are not always available, which may affect buyer decisions.
- 5. Browser Compatibility Variability:** Features and visual layout may render differently across browsers, especially older versions, affecting user experience.
- 6. No Real-Time Order Tracking:** Since payments and order confirmations are manual, there's no instant status update or order tracking feature available.
- 7. Limited Admin Controls:** The admin panel may lack advanced features like bulk uploads, analytics dashboards, or user behavior insights.
- 8. Scalability Concerns:** The current system may not efficiently handle a large number of concurrent users or high volumes of artwork listings without performance issues.
- 9. Security Limitations:** Without secure payment gateways or user authentication best practices (e.g., password hashing, HTTPS), there may be vulnerabilities.
- 10. No AI/Recommendation System:** The platform lacks an AI-driven recommendation engine for personalized artwork suggestions based on user preferences or browsing history.

FUTURE SCOPE

FUTURE SCOPE

As digital transformation accelerates across industries, the online art gallery platform has tremendous potential for growth and enhancement. Future advancements can improve the experience for both artists and buyers, increase accessibility, and expand market reach. Below are the key areas where the application can evolve:

1. Integration of Advanced Technologies

- **Augmented Reality (AR) Previews:** Buyers can use AR to visualize how a piece of art would look in their physical space, enhancing purchase confidence.
- **AI-Based Art Recommendations:** Leveraging AI to analyze user preferences and suggest relevant artworks or artists.
- **NFT and Blockchain Integration:** Introducing digital ownership and authenticity verification using blockchain and NFTs to promote digital art sales.

2. Improved User Experience

- **Live Chat with Artists:** Enabling real-time interaction between buyers and artists for better communication and negotiation.
- **Virtual Exhibitions:** Hosting online art exhibitions or 3D gallery tours to showcase art in immersive environments.
- **Interactive Search Filters:** Enhancing browsing with advanced filters like mood, color palette, medium, or style

3. Improved Security and Trust

- **Verified Artist Badges:** Introducing verification processes to authenticate artists and build buyer trust.
- **Secure Payment Gateways:** Integrating advanced payment systems with fraud protection and multi-currency support.
- **Smart Contracts:** Automating transactions and ownership transfers through smart contracts for transparency and security.

4. Expanded Accessibility

- **Multi-Language Support:** Adding localization features to make the platform usable by global users.
- **Accessibility Features:** Supporting screen readers, alternative text for images, and keyboard navigation to cater to users with disabilities.
- **Mobile App Development:** Creating native Android and iOS apps for smoother access on mobile devices.

5. Better Integration with External Services

- **Social Media Integration:** Allowing easy sharing of artwork on platforms like Instagram, Pinterest, and Facebook to increase visibility.
- **Collaboration with Courier Services:** Integrating with logistics companies for real-time shipping updates and better order tracking.
- **Art Consultancy Integration:** Partnering with art consultants to offer guided services for high-value art purchases.

6. Artist Empowerment Tools

- **Portfolio Builder:** Letting artists create digital portfolios within the platform to showcase their journey and style.
- **Sales Analytics Dashboard:** Offering insights into views, sales, customer interests, and feedback for each artwork.
- **Auction Features:** Introducing bidding functionality to sell artwork through timed or live auctions.

7. Community Building and Events

- **Artist and Buyer Forums:** Enabling discussions, feedback, and collaboration opportunities through community forums.
- **Online Art Competitions:** Hosting contests to promote emerging artists and drive engagement.
- **Newsletter and Content Marketing:** Providing curated newsletters with featured artists, new collections, and industry trends.

8. Industry-Specific Expansion

- **Corporate Art Sales:** Creating a section for businesses to purchase or lease art for office decor and branding.
- **Educational Integration:** Collaborating with art schools and institutions to provide learning content, internships, or student

CONCLUSION

Conclusion

The Artistic Avenue project is a practical implementation of an e-commerce style art gallery that combines both aesthetic appeal and functional depth. It has shown that even with basic tools like PHP and MySQL, robust applications can be developed that handle real-time interactions.

The project demonstrates a full-stack development process, covering frontend design, backend logic, and database connectivity. Moreover, it shows how technology can be harnessed to bring art closer to audiences.

It also acts as a foundational project for future enhancements like mobile responsiveness, payment integration, advanced security, and real-time analytics—skills that are essential for web developers in today's industry.

BIBLIOGRAPHY

Bibliography

- **PHP Documentation**
PHP: Hypertext Preprocessor [Online]. Available at:
<https://www.php.net/docs.php>
(Used for understanding server-side scripting and dynamic web development.)
- **W3Schools – PHP, HTML, and CSS Tutorials**
W3Schools Online Web Tutorials [Online]. Available at:
<https://www.w3schools.com>
(Primary resource for HTML/CSS structure, PHP basics, and form handling.)
- **Bootstrap Framework**
Bootstrap Documentation [Online]. Available at:
<https://getbootstrap.com/docs/5.0/getting-started/introduction/>
(Used for responsive design, grid layouts, and UI components.)
- **MySQL Documentation**
MySQL 8.0 Reference Manual [Online]. Available at:
<https://dev.mysql.com/doc/refman/8.0/en/>
(Referred for creating and managing the backend database system.)
- **Stack Overflow**
Stack Overflow Developer Forum [Online]. Available at:
<https://stackoverflow.com>
(Community-driven solutions and troubleshooting for coding errors.)
- **MDN Web Docs**
Mozilla Developer Network [Online]. Available at:
<https://developer.mozilla.org/>
(Reference for advanced HTML, CSS styling, and JavaScript behavior.)
- **Visual Studio Code**
Visual Studio Code Editor [Software]. Microsoft. Available at:
<https://code.visualstudio.com/>
(Used as the primary IDE for writing and debugging code.)
- **Font Awesome**
Font Awesome Icon Toolkit [Online]. Available at:
<https://fontawesome.com/>
(Icons used for enhancing UI design.)
- **XAMPP Control Panel**
Apache Friends – XAMPP [Software]. Available at:
<https://www.apachefriends.org/index.html>
(Local development environment for PHP and